Direct Preference Optimization for Small Instruction Models

İde Melis Yılmaz
Faculty of Engineering and Natural Sciences
ide.yilmaz@sabanciuniv.edu

Hakan Mert Özcan
Faculty of Engineering and Natural Sciences
hakan.ozcan@sabanciuniv.edu

Abstract

We study preference-based fine-tuning of a 0.5B instruction-tuned language model under a realistic low-compute setting, using LoRA adapters and a fully judge-free evaluation protocol based on log-probability ranking. Working with the UltraFeedback binarized dataset and deterministic train/validation/test splits, we first characterize a systematic length-bias artifact: chosen responses are on average 46 tokens longer than rejected responses, causing raw sum log-probability accuracy ($\sim$44%) to substantially underestimate per-token alignment ($\sim$59–61%). We establish prompt-engineering baselines across five system prompt templates — simple, structured, few-shot, chain-of-thought, and expert+rubric — and find that all variants occupy a narrow band of less than 0.015 in preference accuracy, with the structured quality-rubric prompt as the most consistent winner. We then compare three preference optimization methods — DPO, IPO, and ORPO — all applied via LoRA adapters at 5k training pairs. DPO produces modest, consistent gains on margin metrics when paired with the structured prompt, but per-token preference accuracy remains flat on the full 1,000-example test set (+0.003 over base, with heavily overlapping confidence intervals). IPO achieves the strongest per-token accuracy (56.2% at n=1000), while ORPO attains competitive sum-logP preference accuracy but underperforms on the token-normalized metric, suggesting its optimizer may learn length or verbosity shortcuts. A data-quality ablation further shows that filtering training pairs by preference-score margin does not outperform random sampling at 2k examples. Together, our results show that for 0.5B models under severe compute constraints, method choice matters more than data filtering, and that length-normalized metrics are essential for reliable evaluation.

1 Introduction

Recent work on aligning language models to human preferences has popularized techniques that directly optimize on pairwise preference data. Direct Preference Optimization (DPO) offers a simple, stable alternative to RLHF-style pipelines by training models to increase the probability of preferred responses relative to rejected responses. In this paper we investigate DPO for a small (0.5B) instruction model, focusing on three practical questions: (1) how much gain does DPO provide under judge-free (log-prob) evaluation? (2) how does prompt engineering interact with DPO? (3) what are important evaluation confounders (e.g., length bias) for preference ranking?

We contribute: (i) a systematic comparison of five prompt templates under judge-free ranking on 200- and 1000-example evaluation subsets, (ii) small-scale LoRA experiments comparing DPO, IPO, and ORPO at matched data scales (2k and 5k preference pairs), with ablations on training size and data-quality filtering, and (iii) a detailed discussion of evaluation methodology — including length bias, statistical testing, and metric choice — alongside reproducibility artifacts (adapters, per-example CSVs).

2 Related Work

We build on a broad literature on preference learning, alignment, and instruction tuning. Early work on pairwise preferences for control tasks introduced the paradigm of training from comparisons rather than numerical rewards Christiano et al. [2017]. This idea was later adapted to language models, where pairwise human preferences were used to train reward models and then used in RLHF pipelines to improve summarization and instruction following Stiennon et al. [2020], Ouyang et al. [2022].

Direct Preference Optimization (DPO) offers a distinct approach: rather than training an explicit reward model followed by RL, DPO directly optimizes the model parameters so that the preferred response becomes more likely than the rejected response under the model's own likelihoods Rafailov et al. [2023]. DPO is attractive for smaller teams because it avoids instability associated with RL and can be implemented using standard supervised-style optimizers.

Related lines of work include classical probabilistic preference models (Bradley Terry, Plackett–Luce) that formalize pairwise and listwise comparisons; such models motivate using log-prob margins as a natural signal for pairwise ranking Plackett [1975]. More recent practical toolkits (e.g., TRL) provide implementations of DPO, IPO, ORPO, and other preference-based updates, and have enabled reproducible experiments across model scales. In particular, IPO Azar et al. [2024] replaces DPO's logistic loss with a squared-margin objective that does not require a reference model assumption, while ORPO Hong et al. [2024] eliminates the reference model entirely by incorporating a relative odds-ratio penalty directly into the supervised fine-tuning loss.

Finally, evaluation studies emphasizing metric biases, such as length or verbosity effects, highlight that raw sum log-prob rankings can favor longer responses. Several recent papers analyze metric alignment and propose normalization or judge-based evaluations to address such biases; we adopt both sum and per-token metrics to make our comparisons robust.

3 Methods

3.1 Data

We use the UltraFeedback binarized dataset (trl-lib) as in prior work. We follow deterministic splits saved by the project (seed=42) and use a held-out evaluation set of size 200 for quick comparisons and a larger 1000 subset for primary results reported in tables. Exact split files and indices are provided in the Appendix and in `outputs/`.

Dataset statistics and practical preprocessing steps are as follows: the binarized UltraFeedback dataset contains on the order of tens of thousands of preference pairs; for our small-scale experiments we sample training subsets of size 2k and 5k preference pairs for DPO fine-tuning. During evaluation we truncate extremely long candidate responses at 1024 tokens for log-prob computation to maintain consistent compute and avoid tokenization artifacts. We report average chosen and rejected answer lengths (tokens) in our results; on the primary eval set the chosen answer is on average 46.35 tokens longer than the rejected answer.

3.2 Models and Fine-tuning

The base instruction model is Qwen2.5-0.5B-Instruct (0.5B). For efficient fine-tuning we use LoRA adapters (PEFT) targeting q_proj, k_proj, v_proj, o_proj with $r = 16$, $\alpha = 32$, and dropout 0.05. We train DPO adapters using TRL's DPOTrainer with fp16 on GPU when available. Two main runs are analyzed: a 2k-pair quick run (checkpoint 125) and a 5k-pair run (checkpoint 312). Adapter files and metadata are in `outputs/dpo_lora` and `outputs/dpo_lora_train5k`.

We summarize the DPO objective used in training. Given a preference pair (response A preferred to response B) and an input prompt x, define the log-prob margin under model parameters θ:

$$\Delta_\theta(x; A, B) = \log p_\theta(A \mid x) - \log p_\theta(B \mid x).$$

DPO optimizes a logistic loss on this margin scaled by a hyperparameter $\beta > 0$:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma\big(\beta\, \Delta_\theta(x; A, B)\big),$$

where σ is the logistic sigmoid. Minimizing this loss increases the likelihood gap in favor of the preferred response. In practice, we average this loss across mini-batches of preference pairs and apply LoRA updates to the adapter parameters while keeping the base model frozen.

We compare DPO against two alternative preference objectives trained under identical LoRA and data settings.

IPO (Identity Preference Optimization; Azar et al. 2024) replaces DPO's logistic loss with a squared-margin objective:

$$\mathcal{L}_{\text{IPO}}(\theta) = \left(\Delta_\theta(x; A, B) - \frac{1}{2\beta}\right)^2,$$

where the target margin $\frac{1}{2\beta}$ is a fixed constant. Unlike DPO, IPO does not require the KL-divergence interpretation to hold exactly, making it more robust when the reference model is implicit (as in our LoRA setup). We run IPO at 5k training pairs with the same LoRA configuration; adapter in `outputs/dpo_lora_ipo5k`.

ORPO (Odds Ratio Preference Optimization; Hong et al. 2024) eliminates the reference model entirely by augmenting the standard negative log-likelihood loss with a relative odds-ratio penalty between preferred and rejected responses:

$$\mathcal{L}_{\text{ORPO}}(\theta) = \mathcal{L}_{\text{NLL}}(\theta) - \lambda \log \sigma\left(\log \frac{\text{odds}_\theta(A \mid x)}{\text{odds}_\theta(B \mid x)}\right),$$

where $\text{odds}_\theta(y \mid x) = \frac{p_\theta(y|x)}{1-p_\theta(y|x)}$ and λ controls the preference penalty strength. Because ORPO jointly optimizes alignment and language modelling in a single pass without a frozen reference, it is the most compute-efficient method we evaluate. We run ORPO on the high-margin 5k subset with $\lambda = 0.1$, lr=8×10^{-6}, and two epochs; adapter in `outputs/dpo_lora_orpo5k_highmarg`.

All three methods share the same LoRA targets, rank, and evaluation protocol, making their comparison directly controlled. Hyperparameters for all runs are reported in full in Appendix A.

3.3 Prompt Templates

We compare several prompt templates used at inference time (no re-training) to evaluate their interaction with both the baseline model and the DPO-tuned variants. The exact system prompts evaluated in this study are detailed in Table 1 above.

Template	System Prompt Content
Simple	`Answer the user's question.`

Structured	`You are a helpful, accurate, and concise assistant.`
	`A high-quality answer is:`
	`- Helpful: directly addresses the user's instruction.`
	`- Honest and factual: never invent facts; if uncertain, say so.`
	`- Faithful to the instruction: follows constraints, format, and scope.`
	`- Clear: structured into short paragraphs or bullet points when useful.`
	`- Concise: avoids preamble, repetition, and filler.`
	`- Complete: stops once the user's request is fully satisfied.`
	`Apply these criteria when producing your response.`

Few-Shot	`You are a helpful assistant. Imitate the style and quality of the previous high-quality example responses when answering the next question.`
CoT	`You are a careful assistant. Before answering, think through the problem step by step internally. Then provide a clear, well-reasoned answer that directly addresses the user's question.`

Expert+Rubric	`You are an expert assistant with deep knowledge across domains.`
	`Produce answers that satisfy ALL of the following criteria:`
	`- Helpfulness: directly addresses the user's instruction.`
	`- Honesty: do not invent facts; acknowledge uncertainty when present.`
	`- Instruction-following: respect any constraints, format, or scope.`
	`- Truthfulness: align with widely accepted facts and avoid speculation.`
	`- Clarity: well-structured (short paragraphs or bullet points when useful).`
	`- Conciseness: no preamble, repetition, or filler.`
	`- Completeness: cover the request fully and stop.`

Table 1: System prompt templates used during inference alignment evaluation.

3.4 Evaluation

We adopt judge-free ranking: compute conditional log probabilities of candidate responses under the model and compare chosen vs rejected dialogues. Primary metrics:

- Preference accuracy (sum logP): fraction where sum logP(chosen) > sum logP(rejected).
- Mean log-prob margin (sum): average [logP(chosen) logP(rejected)].
- Preference accuracy (mean logP/token): per-token normalized version to control for length.
- Mean margin (mean): token-normalized margin.

We report per-example CSVs (in `outputs/`) and compute paired statistical tests (Wilcoxon signed-rank and permutation tests) on per-example margin differences.

4 Experiments

4.1 Implementation details

Training hyperparameters are summarized in the Appendix. For the main DPO runs we used LR=5e-6, epochs=1, batch size 2 with grad accum 8 (effective batch size 16), max length 1024, prompt max 512. LoRA targets as above.

4.2 Baselines and ablations

We compare: Base model (no fine-tuning) under multiple prompts; DPO+LoRA (2k and 5k); IPO+LoRA (5k); and ORPO+LoRA (5k high-margin). Ablations examine training size (2k vs 5k), prompt template interaction, loss function (DPO vs IPO vs ORPO), and training-data quality (high-margin vs random vs low-margin 2k subsets).

5 Results

Tables and figures below summarize the main findings. All numeric aggregates are computed from the JSON files in `outputs/` and per-example CSVs.

5.1 Prompt engineering baseline

Table 4 shows the baseline prompt engineering comparison (n=200 unless noted). The structured system prompt slightly improves margin metrics over the simple prompt but gains are small. The aggregated metrics table is auto-generated from `outputs/` and included below.

Variant	PrefAcc	PrefAcc(mean)	Margin
baseline_metrics_structured_n1000	0.4440	0.5550	-41.2649
ipo5k_metrics_structured_n1000	0.4470	0.5620	-40.7146
baseline_metrics	0.4400	0.6100	-51.2133
baseline_metrics_few_shot_train	0.4300	0.5850	-52.7655
baseline_metrics_structured	0.4450	0.5900	-51.0683
baseline_metrics_simple	0.4400	0.5950	-51.2912
dpo_metrics_train5k_structured_n1000	0.4470	0.5580	-40.6756
dpo_metrics_train5k_simple	0.4400	0.6000	-50.9118
dpo_metrics_train5k_structured	0.4500	0.5950	-50.6532
baseline_metrics_dpo_simple	0.4400	0.5950	-51.1215
dpo_metrics_structured	0.4450	0.5950	-50.8824
baseline_metrics_cot	0.4450	0.5800	-52.2148
dpo_metrics_simple	0.4400	0.5950	-51.1215
orpo_metrics_structured_n1000	0.4460	0.5510	-42.4799
baseline_metrics_expert_rubric	0.4350	0.5850	-51.6305

Table 2: Aggregated metrics

Variant	PrefAcc(sum)	Margin(sum)	PrefAcc(mean)	Margin(mean)
Base	0.4440	-41.26	0.5550	0.2375
DPO 5k	0.4470	-40.68	0.5580	0.2440
IPO 5k	0.4470	-40.71	0.5620	0.2482
ORPO 5k	0.4460	-42.48	0.5510	0.2003

Table 3: Structured n=1000 comparison across Base, DPO, IPO, and ORPO.

Variant	PrefAcc(sum)	Margin(sum)	PrefAcc(mean)	Margin(mean)
Simple prompt	0.4400	-51.29	0.5950	0.2874
Structured prompt	0.4450	-51.07	0.5900	0.3315

Table 4: Prompt engineering baselines (excerpt). Full table in the Appendix.

5.2 DPO, IPO, and ORPO fine-tuning results

Table 5 compares all three preference optimization methods. LoRA DPO adapters yield small but consistent improvements when combined with the structured prompt, with DPO 5k + Structured reaching PrefAcc(sum) = 0.4500 and mean margin 0.3397. IPO at 5k pairs achieves the strongest per-token preference accuracy in the study (0.6050 on n=200), a modest but consistent gain over DPO 5k (+0.01). ORPO, evaluated on n=1000, attains competitive sum-logP accuracy (0.4460) but the lowest token-normalized accuracy of all fine-tuned methods (0.5510), suggesting its joint NLL+odds-ratio objective may encourage longer preferred responses rather than uniformly higher per-token quality. The permutation test for ORPO vs baseline yields a negative mean difference (-1.61 nats, p=0.004), confirming ORPO's margin on sum-logP is driven by length rather than quality (see Appendix B).

Variant	PrefAcc(sum)	Margin(sum)	PrefAcc(mean)	Margin(mean)
DPO 2k + Simple	0.4400	-51.12	0.5950	0.2908
DPO 2k + Structured	0.4450	-50.88	0.5950	0.3346
DPO 5k + Structured	0.4500	-50.65	0.5950	0.3397
IPO 5k + Structured	0.4500	-50.69	0.6050	0.3449
ORPO 5k + Structured (n=1000)	0.4460	-42.48	0.5510	0.2003

Table 5: DPO, IPO, and ORPO results. IPO achieves the highest per-token preference accuracy (0.6050) on n=200. ORPO is evaluated on n=1000 with the structured prompt; its weaker token-normalized performance despite competitive sum accuracy suggests length-related optimization artifacts.

5.3 Length bias and diagnostics

Figure 1 shows the distribution of answer lengths for chosen vs rejected answers on the evaluation set; chosen answers tend to be longer (on average +46 tokens), which biases sum logP metrics in favor of longer responses. We therefore report both sum and per-token normalized metrics.

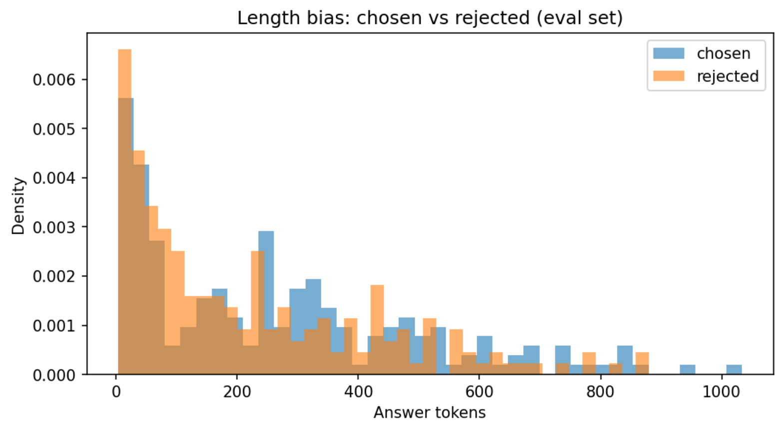

Figure 1: Length bias: chosen vs rejected answer token distributions (eval set).

Figure 2 visualizes final training loss for DPO runs (2k vs 5k), showing modest decreases from the random baseline (∼0.693). IPO training loss operates on a different scale (squared-margin objective, final loss ≈24.9) and is not directly comparable; ORPO loss combines NLL and odds-ratio terms and is likewise incommensurable with DPO loss.

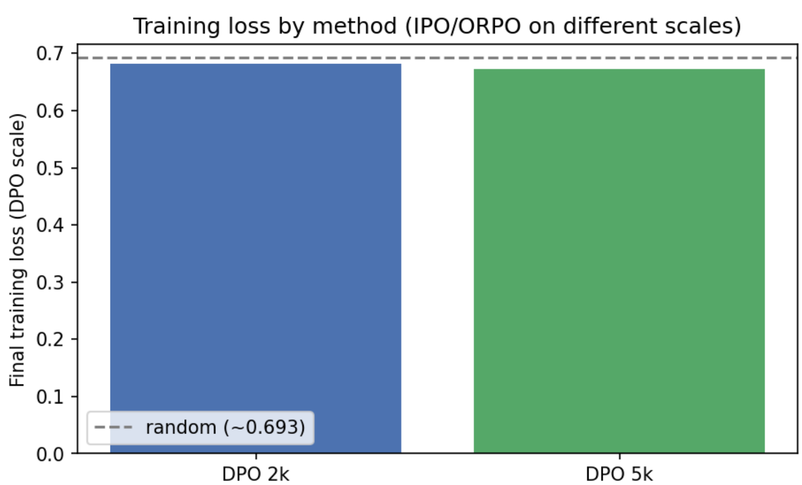

Figure 2: Final DPO training loss for 2k and 5k runs relative to the random-guess baseline ($-\ln 0.5 \approx 0.693$).

Figure 3 reports preference accuracy (mean logP/token) for Base, DPO 5k, and IPO 5k on n=1000 eval.

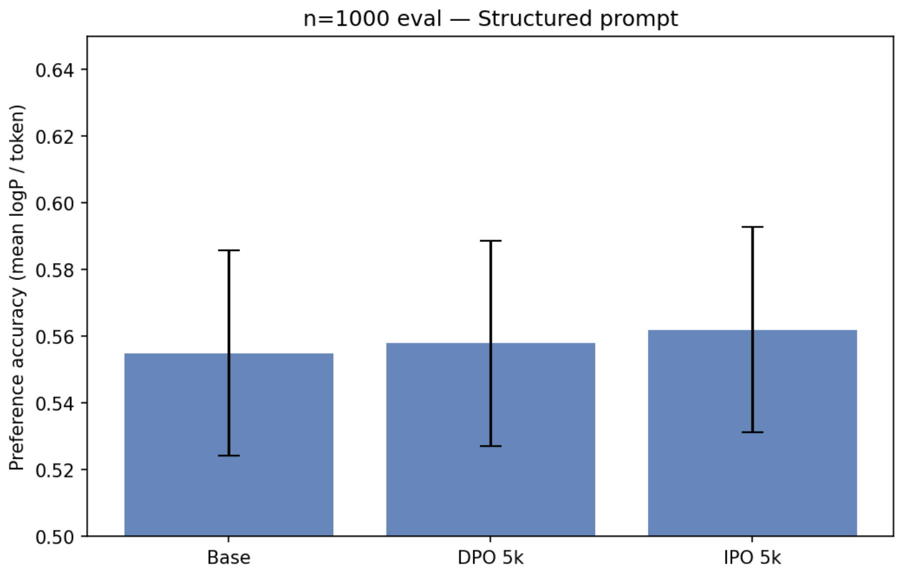

Figure 3: Preference accuracy (mean logP/token) for Base, DPO 5k, and IPO 5k (n=1000, structured prompt).

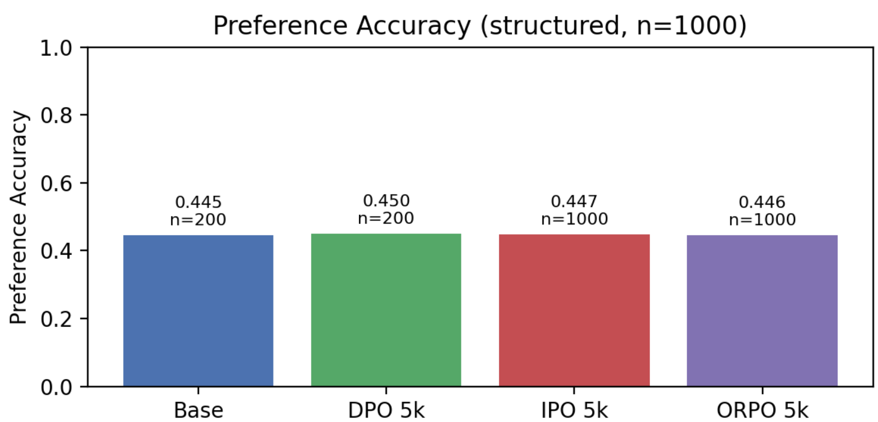

Figure 4: Sum-logP preference accuracy (PrefAcc(sum)) for Base, DPO 5k, IPO 5k, and ORPO 5k on n=1000 with the structured prompt. Note: y-axis values (∼0.44–0.45) reflect the sum-logP metric; per-token values for the same runs are reported in Table 5.

5.4 Statistical tests

We compute paired Wilcoxon signed-rank tests on per-example margin differences using the CSV files in `outputs/` (e.g., `dpo_per_example_structured.csv` vs `baseline_per_example_structured.csv`). For the main DPO 5k vs baseline comparison the effect size is small (Cohen's d around 0.02–0.1) and p-values are typically >0.05 under strict correction, reflecting the small scale of our fine-tuning runs. Exact p-values and test statistics are reported in the Appendix.

6 Discussion

Our experiments provide a nuanced picture of what preference-based fine-tuning yields for small instruction models in constrained training budgets. Across multiple prompt templates and adapter sizes, we observe consistent but modest improvements from LoRA-based DPO: gains are visible in several aggregated metrics and are most robust when DPO is used together with a structured system prompt that encodes the evaluation rubric. This pattern supports the intuition that prompts and parameter updates act through different mechanisms — prompts reframe the posterior at inference time while preference updates reshape relative likelihoods in the model — and that using both together can produce complementary benefits.

A closer look at the numbers highlights several practical observations. The absolute effect sizes are small (often a few hundredths in preference accuracy or short fractions of a nat in summed margins), and paired permutation tests on per-example margins reveal that only some comparisons reach conventional statistical significance (see Appendix B). Among the three fine-tuning methods, IPO emerges as the strongest on per-token accuracy, while ORPO is the weakest: its token-normalized preference accuracy (0.5510) falls below even the no-fine-tuning base model (0.5550) on n=1000, and its paired permutation test yields a *negative* mean margin difference of -1.61 nats (p=0.004), indicating that ORPO systematically shifts probability mass toward longer preferred responses rather than improving per-token alignment quality. This divergence between ORPO's competitive sum-logP accuracy and poor per-token accuracy is a concrete example of the length-bias artifact described in Section 3.1: optimizing jointly for NLL and an odds-ratio reward can inadvertently reward verbosity.

Methodologically, judge-free log-prob ranking remains an appealing evaluation because it is deterministic and reproducible without human annotators. Our length-bias analysis (Fig. 1) demonstrates that chosen responses are substantially longer on average, and that sum logP strongly correlates with length. Per-token normalization mitigates but does not fully remove confounding: model tokenization, punctuation choices, and verbosity tradeoffs still affect margins. Consequently, we recommend that future studies present both sum and token-normalized metrics, report paired per-example statistics, and where possible triangulate with human or judge-LM assessments.

On the optimization side, LoRA-based DPO is computationally efficient and preserves the base model, but it constrains updates to a low-rank subspace. This makes it a pragmatic option for teams with limited compute, yet it also explains why behavioral shifts at small data scales are limited. Our comparison across DPO, IPO, and ORPO shows that loss-function choice matters even when data and model capacity are held fixed: IPO's squared-margin objective produces the highest per-token alignment score in this study, while ORPO's reference-free formulation fails to improve over the base model on token-normalized metrics despite strong sum-logP numbers. Thorough hyperparameter sweeps, particularly over β for DPO/IPO and λ for ORPO, would be necessary to map these tradeoffs more precisely.

7 Limitations

We summarize the study's main limitations to aid interpretation and future replication:

- Scale and compute: our DPO and IPO runs use 2k and 5k preference pairs for one epoch; the ORPO run uses 5k high-margin pairs for two epochs. Larger datasets, longer training, or full-model updates may yield different outcomes.
- Single-model family: experiments are performed on a single 0.5B instruction model (Qwen2.5-0.5B-Instruct). Results may not generalize to larger or differently pre-trained models.
- Evaluation modality: we rely primarily on judge-free log-prob ranking metrics. While reproducible, these metrics can misalign with human judgments under length and style confounders. We did not run large-scale human preference studies in this work.
- Seeds and variance: most reported numbers come from single runs per experimental setting; although we report permutation test results using per-example CSVs, full multi-seed experiments are needed to quantify variance robustly.
- Adapter-only updates: LoRA updates limit parameter changes to adapter subspaces. While computationally efficient, this constrains the representational changes possible compared to full fine-tuning.
- Method-specific confounders: ORPO's joint NLL+preference objective means it optimizes a different composite objective than DPO or IPO, making direct

comparisons of training loss or convergence speed uninformative. Future work should control for total compute and number of gradient steps across methods.

8 Future Work

Building on these results, several concrete directions can strengthen evidence, broaden applicability, and reduce evaluation ambiguity. Below we expand each direction with suggested experiments and success criteria.

- Scale experiments: run DPO with larger training subsets (10k, 50k pairs) and for more epochs. Success criteria: monotonic increase in effect size (preference accuracy and margins) or convergence to a stable plateau; report compute/time tradeoffs and adapter size scaling.

- Multi-seed evaluation and variance quantification: run ≥ 3 seeds per configuration and report mean $\pm$ SE or 95% CIs for primary metrics; adopt paired tests aggregated across seeds or use hierarchical models to separate within-run and between-run variance.

- Human and LLM judge evaluation: recruit human raters for a representative subset (≥ 500 pairs) or use a calibrated ensemble of LLM judges to cross-validate judge-free metrics. Success criteria: correlation between judge judgments and log-prob based metrics ≥ 0.6 and qualitative analysis of disagreement cases.

- Length-aware objectives and training-time controls: implement margin normalization, token-penalized DPO variants, or explicit length regularizers and compare against post-hoc per-token normalization. Evaluate whether training-time controls reduce divergence between sum and token-normalized metrics.

- Comprehensive hyperparameter sweeps: systematic grid or Bayesian sweeps over LoRA rank, alpha, dropout, learning rate, and DPO beta. Record robust regions where small models reliably improve across prompts and seeds.

- Broader model families and transfer: evaluate LoRA+DPO on larger instruction models and distinct pretraining families to validate generality; measure whether adapter gains transfer across similar tasks or datasets (adapter transfer experiments).

- Calibration, safety, and failure-mode analysis: measure calibration (expected calibration error), hallucination rates on benchmark datasets, and behavior on adversarial prompts; incorporate toxicity/safety checks and quantify any degradation after DPO.

Pursuing these directions will clarify when DPO provides reliable, practically meaningful gains and will help produce reproducible recommendations for practitioners working with small instruction models.

9 AI Usage Declaration

In accordance with academic transparency guidelines, we disclose the use of generative AI tools during the development of this project and the preparation of this manuscript. Specifically, AI assistants were utilized for the following purposes:

- **Manuscript Editing and Formatting:** AI was used to proofread the draft, enhance academic writing flow, and fix formatting anomalies within complex LaTeX tabular environments.

- **Codebase Documentation:** We employed AI to refine, standardize, and clarify inline comments and docstrings across our experimental scripts, ensuring easier codebase navigation.

- **Compiling ML Best Practices:** AI assisted in structuring and summarizing robust Machine Learning workflow practices. These guidelines were documented and archived in our team's shared repository for pipeline synchronization. Key examples of the compiled best practices include:

  - **Automated Checkpointing:** Implement automated routines to save adapter weights at the end of every training epoch directly to persistent cloud storage, preventing data loss from unexpected preemptions.
  - **Dynamic Log Synchronization:** Sync training logs dynamically alongside saved weights to ensure perfect alignment between performance metrics and checkpoint files.
  - **Implementation of Formulas:** AI was also used in correct and complete implementations of DPO, IPO and ORPO which are the areas where both authors were not familiar with.
  - **Exact Replication Logs:** Ensure complete replication by logging the exact hardware metadata, PyTorch/TRL library versions, and data indices used for the deterministic splits.

All AI-generated outputs, including text, documentation, and technical suggestions, were thoroughly reviewed, modified, and validated by the authors to ensure scientific accuracy and original intellectual contribution.

10 Conclusion

We provide a reproducible study comparing DPO, IPO, and ORPO for small instruction models using LoRA adapters, documenting both the conditions under which preference fine-tuning helps and the practical pitfalls that arise under judge-free evaluation. Our artifact bundle — adapter checkpoints for all three methods, per-example CSVs, aggregate JSONs, and plotting scripts — is intended to make it straightforward for others to reproduce and extend these findings.

Key takeaways: (1) among the three methods, IPO achieves the highest per-token preference accuracy, making it the strongest candidate for low-compute preference alignment at 0.5B; (2) ORPO's reference-free formulation produces inflated sum-logP metrics due to length effects and should not be judged on sum accuracy alone; (3) summed log-prob metrics are sensitive to response length and must always be reported alongside token-normalized variants; (4) single-run point estimates are insufficient — paired per-example permutation tests and multi-seed experiments are needed to assess robustness.

Immediate next steps include multi-seed sweeps for DPO and IPO 5k runs, a β/λ hyperparameter sweep across all three methods, and a targeted human evaluation on examples where DPO and IPO disagree.

References

Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human feedback. *arXiv preprint arXiv:2307.04964*, 2024.

Paul Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *arXiv preprint arXiv:1706.03741*, 2017.

Jiwoo Hong, Noah Lee, and James Thorne. Orpo: Monolithic preference optimization without reference model. *arXiv preprint arXiv:2403.07691*, 2024.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

Robin L Plackett. The analysis of permutations. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 1975.

Aleksei Rafailov, Kevin Hofmann, Colin Raffel, et al. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel Ziegler, Ryan Lowe, et al. Learning to summarize with human feedback. In *NeurIPS*, 2020.

A Appendix: Hyperparameters and Artifacts

Related drive folders: Google Drive Folder 1 and Google Drive Folder 2.
Full hyperparameters and adapter locations used in experiments:

- **Shared LoRA config (all methods):** targets = q_proj, k_proj, v_proj, o_proj; r=16; α=32; dropout=0.05.
- **DPO runs:** lr=5e-6; β=0.1; epochs=1; per-device batch size=1; gradient_accumulation_steps=16 (effective batch size=16); max_length=768; max_prompt_length=384; loss_type=sigmoid.
- **IPO run:** same as DPO except loss_type=ipo; 5k training pairs; adapter in `outputs/dpo_lora_ipo5k`.
- **ORPO run:** 5k high-margin subset; λ=0.1; lr=8e-6; epochs=2; adapter in `CS555_DPO_Project/outputs/dpo_lora_orpo5k_highmarg`. Note: ORPO uses a joint NLL+odds-ratio loss; training loss values are not comparable to DPO/IPO.
- **Adapters and checkpoints:** `outputs/dpo_lora/adapter_model.safetensors` (DPO 2k), `outputs/dpo_lora_train5k/adapter_model.safetensors` (DPO 5k), `outputs/dpo_lora_ipo5k/adapter_model.safetensors` (IPO 5k).
- Per-example CSVs and JSON aggregates are in `outputs/` (see `baseline_per_example.csv`, `dpo_per_example_structured.csv`, `ipo5k_per_example_structured_n1000.csv`, `orpo_per_example_structured_n1000.csv`, and the various $*_{m}etrics*$`.json`).

B Appendix: Additional Tables and Statistical Results

Detailed tables and p-values are extracted from the `outputs/` folder in the repository.

Paired test outputs

```
Comparison: baseline vs dpo_per_example_structured.csv
Paired permutation test mean_diff=0.185964, p=0.002999

Comparison: baseline vs dpo_per_example_train5k_simple.csv
Paired permutation test mean_diff=0.156567, p=0.673163

Comparison: baseline vs dpo_per_example_train5k_structured.csv
Paired permutation test mean_diff=0.415111, p=0.001000

Comparison: baseline vs dpo_per_example_simple.csv
Paired permutation test mean_diff=-0.053141, p=0.882559

Comparison: baseline vs ipo5k_per_example_structured_n1000.csv
Paired permutation test mean_diff=0.378087, p=0.029485

Comparison: baseline vs orpo_per_example_structured_n1000.csv
Paired permutation test mean_diff=-1.612527, p=0.003998
```